

Computed but Not Written: Where a Misleading Identifier Enters a Code Trace

Anonymous ACL submission

Abstract

Misleading identifiers lower the accuracy of language models on code, and step-by-step reasoning does not restore it. We ask where the identifier enters. We construct a cue-conflict version of output prediction: short Python functions in which one identifier implies an executable value different from the one the code computes, rendered with congruent, neutral and misleading names on matched twins, and traced line by line by seven instruction-tuned models with three sets of worked examples. When a variable computed as `len(xs)` carries a sum-like name, general models write the sum at that step +12.4 to +58.4 points above the twin’s rate, and a code model does the same on `max(xs)`. Linear probes read the code’s value from the token at which the model writes the step in 85–100% of these instances and the name’s value in 2%; activation patching places the origin at the name’s tokens and shows the influence arriving at the writing position in the middle of the network, where a single-layer replacement removes it. Writing the expression beside the value in the worked examples removes the effect. It is several times weaker in a REPL transcript, reduced to a residue at 34B parameters, and not detectable on renamed CRUXEval functions. A trace can hold the right value and write the wrong one.

1 Introduction

Consider the function in Figure 1(a). The variable `sum_all` is assigned `len(xs)`, so on the input `[5, 3]` it holds 2 and the call returns 9. The name says that it holds `sum(xs)`, which would be 8 and would make the call return 15. Both readings are executable, and only one is the code. We call 8 the *lure value* of the name and 2 its *true value*. If a model writes 8 for that step, or 15 as the answer, the meaning of the name has reached the computation.

Motivation. Identifiers that contradict the code they name reduce model accuracy on code comprehension (Le et al., 2025; Lam et al., 2025), and Le et al. (2026) report that the loss is largest for the strongest code models and is not removed by step-by-step reasoning. A drop in accuracy does not say where the name interferes: the input may be misread, the computation may be corrupted, or a correct computation may be misread at the output. Kudo et al. (2026) locate, with probes and patching on arithmetic, the point in a chain of thought at which each intermediate value becomes decodable, and Anonymous (2026) add a cue conflict to that setting and find that a written chain removes the influence of a misleading variable name wherever the value is represented. Code traces differ from arithmetic chains in one respect that turns out to be decisive: a trace writes a variable’s value without restating the expression that produced it.

This paper. We place two cues in conflict, as in the Stroop task (Stroop, 1935) and the shape–texture experiments on vision models (Geirhos et al., 2019), inside a CRUXEval-style output-prediction task (Gu et al., 2024) on synthetic Python. The lure is a program rather than a word: each misleading name comes from a table of (name, operation) pairs, and the lure value is obtained by executing the function with the implied operation in place of the real one. Matched twins differ in a single identifier, so every number below is an excess over the neutral twin. The name’s influence turns out to be confined to one kind of step, one kind of name and one kind of format, and we locate it: the value is computed and decodable where the model writes the step, the name’s tokens are the causal origin, and the output head writes the name’s value anyway.

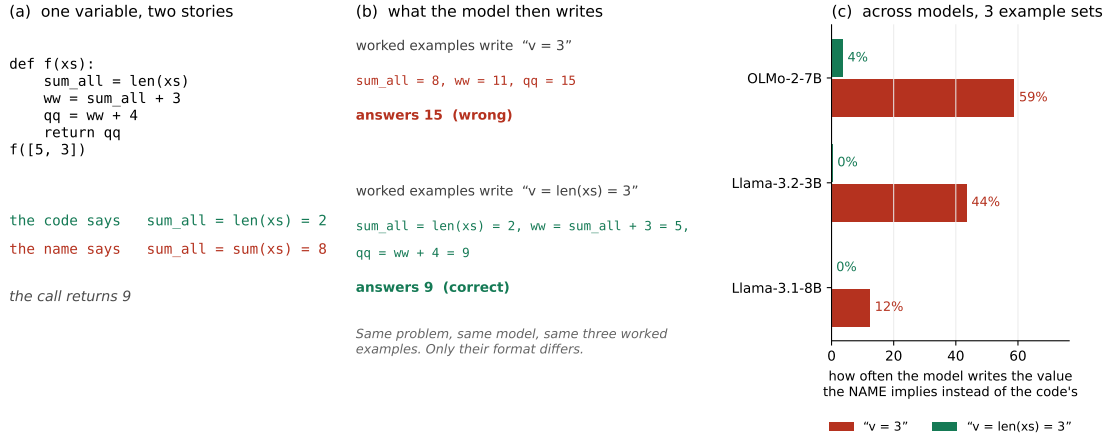


Figure 1: (a) One function under two readings of its first variable. (b) What OLMo-2-7B writes for it under two formats of the same three worked examples. (c) How often three models write the name’s value instead of the code’s at that step, for variables computed as `len(xs)` and named like a sum, mean over three sets of worked examples; red when the examples write `v = 3`, green when they write `v = len(xs) = 3`.

2 Setup

Task. A problem is a Python function of one list argument, a chain of one to three assignments, a return, and a call on a short list of distinct digits. The first assignment applies a list aggregate (`len`, `sum`, `max`, `min`); later ones apply a unary or binary integer operation. Every intermediate and final value is a single digit, as in Kudo et al. (2026), so probes are 10-way classifiers. Table 3 in the appendix lists the levels. Our main level has three assignments (level 5); the one-aggregate level (3) is reported in the appendix.

Naming conditions. Each problem is generated once and rendered with different names for one variable, the *target*: a *congruent* name whose implied operation is the code’s own, a *neutral* name of one or two letters, and a *misleading* name whose implied operation differs. Names come from a fixed table of 25 identifiers in eleven families (`count`, `n_items`, `length` for `len`; `total`, `sum_all`, `acc` for `sum`; and so on; Table 4). The lure value is computed by execution and must differ from the true value and from every other value in the program and its input, so a lure answer cannot be correct for another reason and cannot be copied from the prompt. This last rule removes `max` and `min` names as lures on a list-defined variable, since their implied value is an element of the input. The testable misleading pairs on the aggregate step are therefore a `len`-, `max`- or `min`-computed variable named like a sum, and a `sum`-, `max`- or `min`-computed variable named like a

length. A name is a lure for a model only if that model reads it as the intended operation; a log-probability gate records which operation each name evokes for each model (Appendix B).

Formats. In the *direct* regime the model writes only the return value after three worked examples. In the *trace* regime the three examples trace the execution as `qq = 3`, `v = 6`, `vv = 9` and then answer, and the model does the same for the problem. Three further formats change only what a step looks like: *trace-expr* writes `qq = len(xs) = 3`; *REPL* writes `>>> qq` and then 3; *comment* rewrites the program with each value as an end-of-line comment. A *prose* regime asks the model to think step by step in its own words with no examples. Every few-shot result is the mean over three fixed sets of worked examples.

Measures. Our main measure is the *written-lure excess*: the rate at which the trace’s step for the target states the lure value, minus the rate at which the neutral twin’s step for the same variable states that same number, so that an arbitrary wrong number counts in both. We count an effect as reliable when its sign is the same under all three sets of worked examples and a cluster bootstrap over matched problems excludes zero, and we treat effects inside two points as equivalent to zero.

Probes and patching. A linear probe is trained on neutral problems only, one per layer and position, to read the target’s value from the residual stream, with the recipe and hyperparameters of Kudo et al. (2026) and a control probe keyed

on the name’s identity (Hewitt and Liang, 2019). We test it on misleading problems at the *decision token*, the = after which the trace writes the target’s value, the first step of the trace. To test cause, we replace the residual stream at one layer with the matched neutral twin’s, either at every token of the name or at the decision token, and read the digit the model writes (Zhang and Nanda, 2024); a neutral-for-neutral replacement bounds the damage of any patch.

Models. Seven instruction-tuned models (Table 1) on every behavioural experiment, with 1,000 matched problems per cell and set of worked examples; CodeLlama-34B on the main level; probes and patching on the first three.

3 Results

Behaviour. Table 1 gives the main level. Without a trace, the name moves the answer by -0.2 to +3.1 points, and the excess is reliable for 3 of 7 models. With a trace, the name enters at one step. A variable computed as `len(xs)` and named `total`, `sum_all` or `acc` is traced as the sum +58.4 points above the twin’s rate by OLMo-2-7B, +43.4 by Llama-3.2-3B and +12.4 by Llama-3.1-8B, whose neutral traces are 95.7, 97.4 and 99.6% accurate. The code models do this on `max(xs)` instead: CodeLlama-7B writes the sum for a `max`-computed variable +10.7 points above its twin, and CodeGemma-7B +10.8 points at level 3. The reverse pairing is weaker: a `sum`-computed variable named like a length is traced as the length within two points of zero at this level for every model above the floor, and +4.4 points above it for OLMo-2-7B at level 3; no other pairing moves any step (Table 5). Gemma-3-4B shows nothing, and it is one of the 2 models whose gate reads most sum-family names as `len`; the other, OLMo-2-1B, is at 37.8% neutral accuracy and moves a few points in every direction.

The sum is not a default. When a neutral trace gets a `len` step wrong, it writes the maximum in 29–60% of cases and the length off by one in 60–73%; it writes the sum in 14–32%. The sum appears when the name says `sum`.

The format causes it. The trace format writes a value without its expression. Table 2 holds problems, models and example sets fixed and changes the format. Writing `qq = len(xs) = 3` in the examples lowers the written-lure rate by 55, 43 and

12 points, paired over instances, and raises accuracy in the cell from 95.7 to 98.5% for OLMo-2-7B; on every other (operation, name) pairing the rate is already near zero and stays there, so this is not a general gain. The REPL transcript, which also writes values without expressions, shows 6 and 3 times less than the trace, and nothing for Llama-3.1-8B; we had predicted it within a factor of two. Values as comments on the code remove the effect for two models and leave +6.8 points for Llama-3.2-3B. Prose reasoning never shows it (appendix).

The value is there. Probes trained on neutral traces and tested on the misleading cell read the code’s value at the decision token in 85%, 90% and 100% of instances at the best layer, rising through the network (Figure 2, green). The name’s value is read in at most 2% at any layer. The same models write the sum in 59%, 44% and 12% of these instances. The computation is intact in the residual stream and the output head does not use it. The control probe scores 1.00 at this position for two models and 0.80 for the third, so the name is also decodable there; the value probe cannot be reading it, since name and value are independent in its training data.

Where it comes from and where it acts. Figure 2 shows the two patching sites. Replacing the name’s tokens at layer 0 alone removes 97% and 100% of the lure writes at 2% damage, and the removal decays to below 10% by layers 17 and 15. Replacing the decision token alone removes half of the writes from layers 15, 10 and 8, 90% from layers 16, 12 and 9, and 97–100% at every layer at once, with 2% damage or less. The neutral-for-neutral control changes at most 1.0% of outputs at either site. The two curves cross where the name’s contribution has moved from its own tokens into the decision token, where the probe reads the code’s value at those same layers. The twin’s vector there also holds the code’s value; substituting it removes the sum and nothing else.

Boundaries. Three pre-registered tests bound the effect. On a level whose middle step is a unary operation, no family of names moves that step for any model above the floor (largest excess +0.6). CodeLlama-34B traces the cell of Table 1 with a written-lure excess of +0.0 and the `max` cell with +1.8 [+1.0, +2.8], inside our equivalence margin; we had predicted it to behave like CodeLlama-7B.

Model	Trace acc.	Direct: lure	Trace: len→sum	Trace: max→sum
OLMo-2-7B	95.7	+0.4	+58.4* [+55.1, +61.8]	+1.9 [+1.1, +2.9]
Llama-3.2-3B	97.4	+0.8	+43.4* [+40.0, +46.7]	+0.0 [+0.0, +0.0]
Llama-3.1-8B	99.6	+0.5	+12.4* [+10.2, +14.6]	+0.0 [+0.0, +0.0]
CodeLlama-7B	91.7	+1.0	+3.7* [+2.6, +5.0]	+10.7* [+8.5, +12.8]
CodeGemma-7B	99.2	+1.5*	+0.1 [+0.0, +0.4]	+0.0 [+0.0, +0.0]
Gemma-3-4B	99.1	+0.8	+0.0 [+0.0, +0.0]	+0.0 [+0.0, +0.0]
OLMo-2-1B	37.8	+3.1*	+3.7* [+2.5, +5.1]	+4.2* [+2.8, +5.7]
CodeLlama-34B	98.9	—	+0.0 [+0.0, +0.0]	+1.8* [+1.0, +2.8]

Table 1: Level 5, mean over three sets of worked examples. *Trace acc.*: accuracy on neutral problems. *Direct: lure*: lure answers above the twin’s rate, queried variable. Last two columns: written-lure excess at the aggregate step when a variable computed as `len(xs)` or `max(xs)` carries a sum-like name, with 95% bootstrap intervals. *: same sign under all three sets and interval excluding zero.

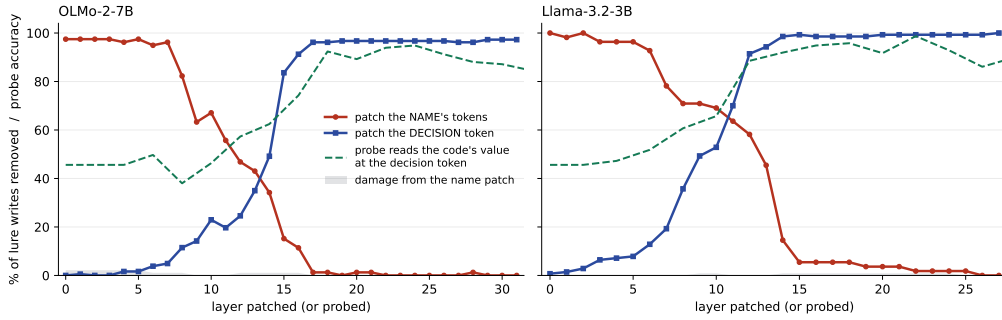


Figure 2: Proportion of lure writes removed by replacing the residual stream with the neutral twin’s at one layer, at every token of the name (red) or at the decision token (blue), for the cell of Table 2; the probe’s accuracy for the code’s value at the decision token (green, dashed); damage to correct instances from the name patch (grey). Llama-3.1-8B’s lure writes fall on the two-token name `sum_all`, which has no token-aligned neutral twin, so its name-site curve is missing rather than flat.

Model	trace	REPL	comment	trace-expr
OLMo-2-7B	+58.4*	+9.5*	+0.0	+3.4*
Llama-3.2-3B	+43.4*	+12.6*	+6.8*	+0.4
Llama-3.1-8B	+12.4*	+0.2	+0.1	+0.0

Table 2: The cell of Table 1 under four formats of the same worked examples (intervals in Table 6).

On 89 CRUXEval functions that assign a length, a loop counter or a maximum, renamed by AST rewriting to `v` or `total` and verified by execution, every interval for the paired misleading-minus-neutral accuracy covers zero, and none extends beyond 10 points in the direct regime or 13 points in prose (Table 7). Natural code has no analogue of the trace format, so this bounds the regimes people use.

4 Related Work

Kudo et al. (2026) established when intermediate values are computed during arithmetic chains, and Anonymous (2026) found the chain protective against a misleading name wherever the value is

represented; ours is the case in which the value is represented and the trace still writes the name’s. Misleading identifiers lower accuracy on real code (Le et al., 2025; Lam et al., 2025; Le et al., 2026; Orvalho and Kwiatkowska, 2025; Gao et al., 2023) without localisation inside the model. Chain faithfulness has been measured by perturbing prompts and chains (Turpin et al., 2023; Lanham et al., 2023; Chen et al., 2025); we measure it at the token where the chain commits to a value. The Stroop design has been applied to next-token prediction (Wang, 2026; Hu et al., 2026), not to a step inside a chain.

5 Conclusion

In a terse trace, small models write the value a sum-like name implies for a variable the code computes otherwise, while the code’s value is decodable at the token that writes the step. The name’s tokens are the origin, the decision token is where the influence acts, and one replacement at either removes it, as does showing the expression beside the value.

Limitations

The programs are synthetic and short, with single-digit values, so the internal analysis has a clean ground truth at the cost of realism; the renamed CRUXEval functions are the only natural code, and only in regimes where the effect does not occur, with power to exclude effects above about five points. The effect depends on the worked-example format, and the format that shows it is one we constructed; the REPL and comment formats are closer to what people write and show much less. The models are at most 34B, and the largest shows only a residue; two of four pre-registered predictions about the boundaries were wrong, which we report as such. The name-site patch cannot be applied to two-token names with token-aligned twins, so one model’s name-site result is missing. Probes read linearly decodable information only, and patching a single position cannot exclude side effects, which is why a neutral-for-neutral control accompanies every intervention. Models of Chinese origin were outside the scope of this study, which leaves out the strongest open code models of Le et al. (2026).

References

- Anonymous. 2026. Name or value? cue conflict reveals when chain of thought overrides lexical priors. Under review.
- Yanda Chen et al. 2025. Reasoning models don’t always say what they think. *arXiv preprint arXiv:2505.05410*.
- Shuzheng Gao, Cuiyun Gao, Chaozheng Wang, Jun Sun, David Lo, and Yue Yu. 2023. Two sides of the same coin: Exploiting the impact of identifiers in neural code comprehension. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, pages 1933–1945. ArXiv:2207.11104.
- Robert Geirhos, Patricia Rubisch, Claudio Michaelis, Matthias Bethge, Felix A. Wichmann, and Wieland Brendel. 2019. ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness. In *International Conference on Learning Representations*.
- Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. 2024. CRUXEval: A benchmark for code reasoning, understanding and execution. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.
- John Hewitt and Percy Liang. 2019. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*.
- Xiaoyang Hu, Mike Angstadt, Shane Storks, Zan Huang, Aman Taxali, Alex Weigard, Richard L. Lewis, and Chandra Sripada. 2026. Conflict and congruency effects in large language models: In-weight and in-context competition in a verbal conflict task. *arXiv preprint arXiv:2608.11510*.
- Keito Kudo, Yoichi Aoki, Tatsuki Kuribayashi, Shusaku Sone, Masaya Taniguchi, Ana Brassard, Keisuke Sakaguchi, and Kentaro Inui. 2026. LLMs faithfully and iteratively compute answers during CoT: A systematic analysis with multi-step arithmetics. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 1114–1153. Code: <https://github.com/keitokudo/faithful-cot-multistep-arithmetic>.
- Man Ho Lam, Chaozheng Wang, Jen-tse Huang, and Michael R. Lyu. 2025. CodeCrash: Exposing LLM fragility to misleading natural language in code reasoning. In *Advances in Neural Information Processing Systems*, volume 38. ArXiv:2504.14119.
- Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, Kamilė Lukošiuūtė, Karina Nguyen, Newton Cheng, Nicholas Joseph, Nicholas Schiefer, Oliver Rausch, Robin Larson, Sam McCandlish, Sandipan Kundu, and 11 others. 2023. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*.
- Cuong Chi Le, Minh V. T. Pham, Cuong Duc Van, Hoang N. Phan, Huy N. Phan, and Tien N. Nguyen. 2025. When names disappear: Revealing what LLMs actually understand about code. *arXiv preprint arXiv:2510.03178*.
- Jack Le, Anh H. N. Nguyen, and Tien N. Nguyen. 2026. Do machines struggle where humans do? LLM and human comprehension of obfuscated code. *arXiv preprint arXiv:2606.31725*. Cite in the third person under [review].
- Pedro Orvalho and Marta Kwiatkowska. 2025. Are large language models robust in understanding code against semantics-preserving mutations? *arXiv preprint arXiv:2505.10443*.
- J. Ridley Stroop. 1935. Studies of interference in serial verbal reactions. *Journal of Experimental Psychology*, 18(6):643–662.
- Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. 2023. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, volume 36, pages 74952–74965.

386 Han-yu Wang. 2026. Persistent priors, preserved tar-
387 gets: A Stroop-style paradigm for lexical override.
388 *arXiv preprint arXiv:2606.07555*.

389 Fred Zhang and Neel Nanda. 2024. Towards best prac-
390 tices of activation patching in language models: Met-
391 rics and methods. In *International Conference on*
392 *Learning Representations*.

Level	Neutral program (body)	Traced steps
1	<code>v = len(xs); return v</code>	1
2	<code>v = len(xs); w = v * 2; return w</code>	2
3	<code>v = len(xs); w = max(xs); u = v + w; return u</code>	3
4	level 3 with an unused fourth assignment	3
5	<code>v = len(xs); w = v * 2; u = w + 3; return u</code>	3
6	level 5 with the misleading name on the middle step	3

Table 3: The levels, one example each. The aggregate in the first statement is any of `len`, `sum`, `max`, `min`; unary operations are `*`, `//2`, `+`, `-`. The misleading name sits on the first statement at levels 1–5 and on the second at level 6.

Family	Names	Implied operation
<code>len</code>	<code>count</code> , <code>n_items</code> , <code>length</code>	<code>len(xs)</code>
<code>sum</code>	<code>total</code> , <code>sum_all</code> , <code>acc</code>	<code>sum(xs)</code>
<code>max</code>	<code>largest</code> , <code>max_val</code> , <code>peak</code>	<code>max(xs)</code>
<code>min</code>	<code>smallest</code> , <code>min_val</code> , <code>low</code>	<code>min(xs)</code>
<code>double</code>	<code>double</code> , <code>twice</code>	<code>2 * a</code>
<code>half</code>	<code>half</code>	<code>a // 2</code>
<code>succ</code>	<code>succ</code> , <code>next_val</code>	<code>a + 1</code>
<code>pred</code>	<code>pred</code> , <code>prev_val</code>	<code>a - 1</code>
<code>add</code>	<code>combined</code> , <code>both</code>	<code>a + b</code>
<code>sub</code>	<code>diff</code> , <code>gap</code>	<code>a - b</code>
<code>mul</code>	<code>product</code> , <code>scaled</code>	<code>a * b</code>

Table 4: The lure table. The gate (Appendix B) scores each name by the log-probability each model assigns to the candidate right-hand sides; 12 of 25 names are read as intended by at least 80% of models. The `min` and `max` names cannot be lures on the aggregate step (their value is an input element) and are used as congruent and alternative-lure names.

B Lure gate

394

A name is a lure for a model only if that model reads it as the intended operation. We score, for each model, the log-probability of each candidate right-hand side after `def f(xs):\n total =`, and record which operation each name evokes (Table 4). 12 of the 25 names are read as intended by at least 80% of the models; most `sum`-family names are read as `len` rather than `sum` by 2 of the seven models.

395

396

397

398

C Full matrix of pairings

399

Model	<code>len</code> → <code>sum</code>	<code>max</code> → <code>sum</code>	<code>min</code> → <code>sum</code>	<code>sum</code> → <code>len</code>	<code>max</code> → <code>len</code>
OLMo-2-7B	+17.6* / +58.4*	+0.9 / +1.9	+0.0 / +0.2	+4.4* / +0.0	+0.0 / +0.0
Llama-3.2-3B	+23.6* / +43.4*	+1.1 / +0.0	+0.0 / +0.0	+0.0 / +0.0	+0.0 / +0.0
Llama-3.1-8B	+1.5 / +12.4*	+0.7 / +0.0	+0.0 / +0.0	+0.0 / +0.0	+0.0 / +0.0
CodeLlama-7B	+0.9 / +3.7*	+7.5 / +10.7*	+1.0 / +1.7	+1.5 / +0.0	+0.9* / +0.6
CodeGemma-7B	+0.0 / +0.1	+10.8* / +0.0	+0.0 / +0.0	+0.0 / +0.0	+0.0 / +0.0
Gemma-3-4B	+0.0 / +0.0	+0.2 / +0.0	+0.0 / +0.0	+0.0 / +0.0	+0.0 / +0.0
OLMo-2-1B	+1.2 / +3.7*	+2.8* / +4.2*	+0.3 / -0.2	+10.6* / +11.1*	+9.6* / +5.6*

Table 5: Written-lure excess at the aggregate step, trace regime, level 3 / level 5, three sets of worked examples pooled; * as in Table 1.

D Formats with intervals

Model	trace	REPL	comment	trace-expr
OLMo-2-7B	+58.4* [+55.2, +61.6]	+9.5* [+7.5, +11.5]	+0.0 [+0.0, +0.0]	+3.4* [+2.2, +4.6]
Llama-3.2-3B	+43.4* [+40.2, +46.8]	+12.6* [+10.4, +14.9]	+6.8* [+5.3, +8.5]	+0.4 [+0.0, +0.8]
Llama-3.1-8B	+12.4* [+10.3, +14.7]	+0.2 [+0.0, +0.6]	+0.1 [+0.0, +0.4]	+0.0 [+0.0, +0.0]

Table 6: Table 2 with 95% bootstrap intervals over matched problems.

E Renamed CRUXEval

Model	Direct acc.	Direct Δ	Prose acc.	Prose Δ
OLMo-2-7B	10	+1.2 [+0.0, +3.6]	22	-1.2 [-9.5, +8.3]
Llama-3.2-3B	19	+1.2 [-2.4, +6.0]	25	+3.6 [-4.8, +11.9]
Llama-3.1-8B	22	-3.6 [-9.5, +2.4]	40	+0.0 [-9.5, +9.5]
CodeLlama-7B	18	+2.4 [+0.0, +6.0]	13	+2.4 [-3.6, +8.3]
CodeGemma-7B	18	+0.0 [-4.8, +4.8]	22	+2.4 [-8.3, +13.1]
Gemma-3-4B	15	-2.4 [-6.0, +0.0]	45	+4.8 [-1.2, +11.9]
OLMo-2-1B	16	-3.6 [-8.3, +0.0]	3	+6.0 [+0.0, +13.1]

Table 7: 89 CRUXEval functions that assign a variable from `len(. . .)` (48), a loop counter (38) or `max(. . .)` (3), of which 84 use it downstream; each rendered with the original name, the neutral name `v`, and the misleading name `total`. Accuracy on the neutral rendering and the paired misleading-minus-neutral difference in points with a bootstrap interval over functions.